

Addressing Secure Authentication Challenges in Embedded Systems

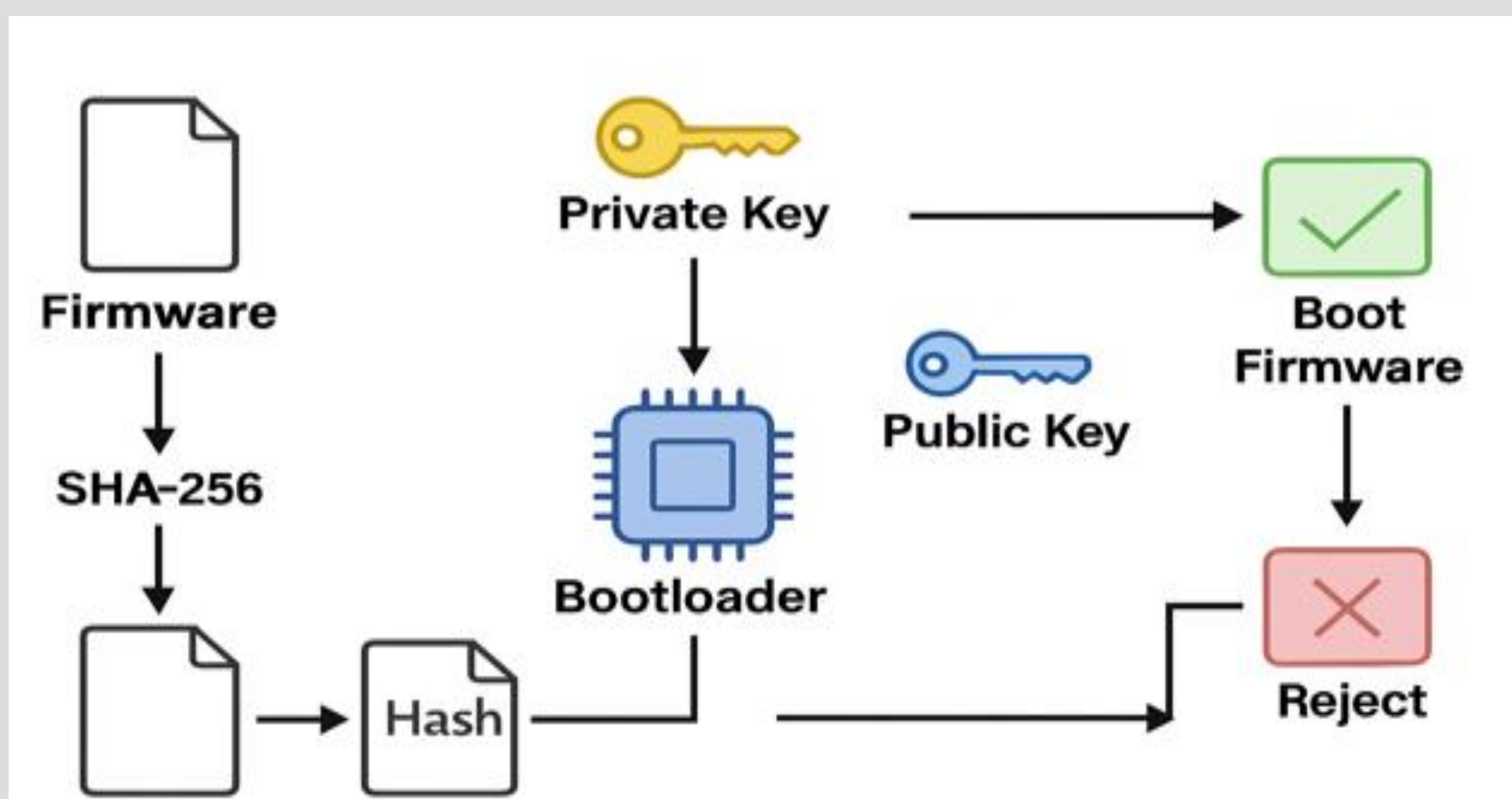
Nishant Chinnasami and Dr. Rasha Karakchi
Department of Computer Science and Engineering

ABSTRACT

- ❑ The primary objective of this project is to enhance security, reliability, and trust in embedded systems and FPGA-based platforms.
- ❑ We address vulnerabilities such as firmware tampering and malicious code execution by developing a secure bootloader that verifies the integrity and authenticity of firmware before execution. We use SHA-256 and digital signatures to prevent firmware tampering.
- ❑ FPGAs are reconfigurable devices used in aerospace, military, industrial control, and IoT applications.
- ❑ *Traditional* FPGA bootloaders lack authentication mechanisms and blindly trust firmware from memory.
- ❑ Malicious firmware can alter device behavior, leak sensitive data, or render systems non-functional.

RSA OVERVIEW

- RSA is a public-key cryptosystem used for authentication. The private key (kept secure) signs the firmware. The bootloader stores only the public key and verifies the firmware signature.



RESOURCES

1. Wang, X., Zhang, T., & Song, J. (2015). FPGA-based System Security and Authentication Mechanisms.
2. Guo, Y., & Zhang, M. (2019). FPGA Bitstream Security: Protecting Configuration Data.
3. National Institute of Standards and Technology (NIST). (2013). Digital Signature Standard (DSS).

FUTURE WORK

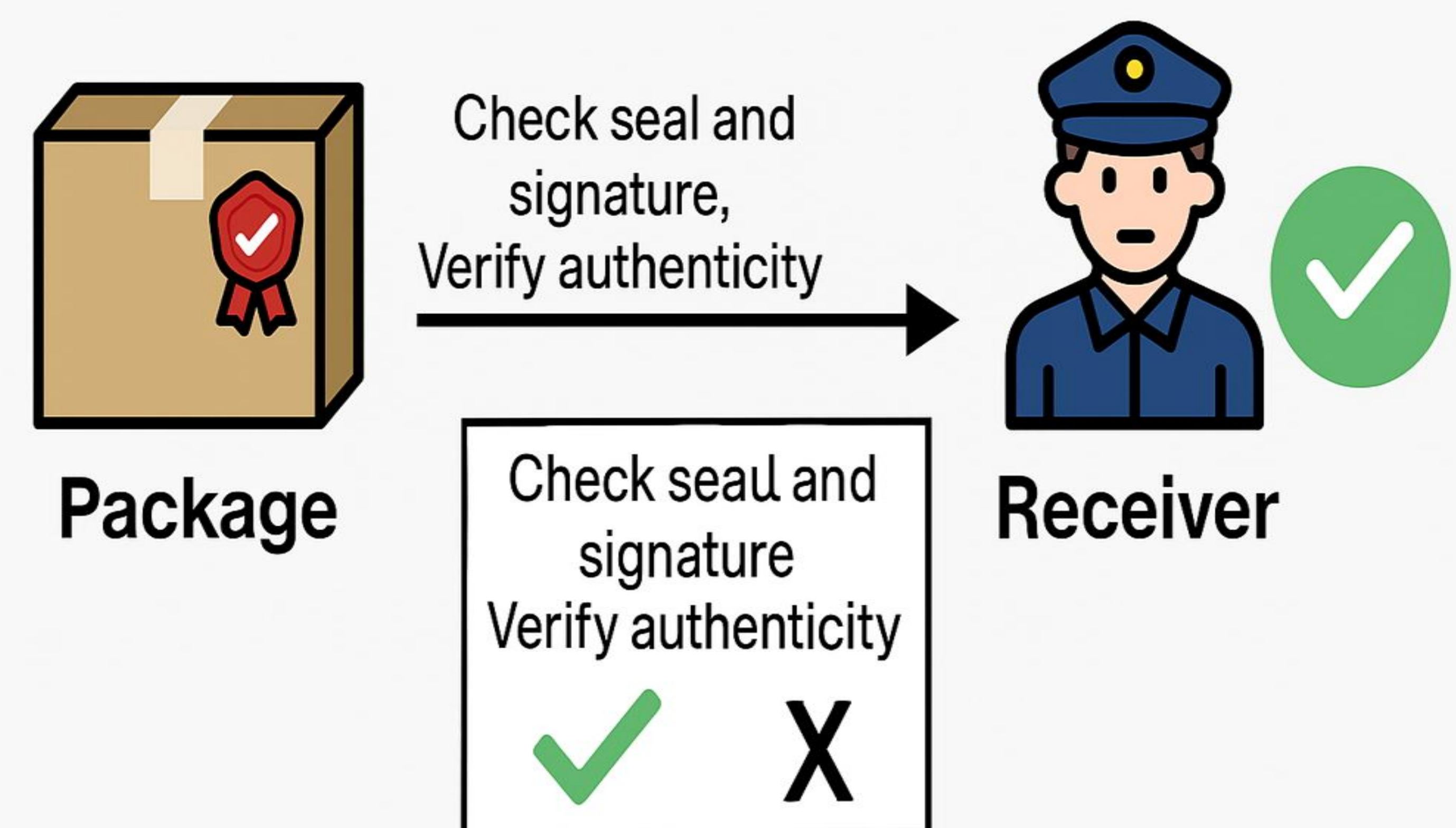
- Develop a hardware-based booting system
- Compare with the software-based booting
- Testing on real-time datasets

THREATS TO FIRMWARE

- **Trojan Horse Insertion:** Inserting malicious logic during firmware development or updates.
- **Bitstream Tampering:** Modifying the configuration file to change FPGA behavior.
- **Reverse Engineering:** Extracting functionality from firmware to exploit vulnerabilities.
- **Unauthorized Firmware Updates:** Installing unofficial or unsigned firmware.

Secure Package Delivery Analogy

Securing FPGA Bootloader



PROPOSED WORK

Our solution uses a secure bootloader that:

- Loads firmware from SPI Flash
- Computes a SHA-256 hash of the firmware
- Verifies a digital signature using RSA public key
- Boots only verified firmware

ACKNOWLEDGMENTS

The printing of this poster was funded by the Department of Computer Science and Engineering at the University of South Carolina. This project was submitted for the Magellan Scholar Program.

1. Generate RSA Key Pair(2048-bit)

2. Export Public Key

3. Verify Key Validity

4. Sign Firmware Hash (SHA-256) with Private Key

5. Verify Digital Signature with Public Key



**Molinaroli College of
Engineering and Computing**
UNIVERSITY OF SOUTH CAROLINA